Harsh Kumar **D15B/65**

Experiment No. 8

Aim : Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Theory :

1. Dataset Source

We use the **MNIST Handwritten Digits Dataset**:

Dataset Link:
https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. Dataset Description

MNIST is a widely used dataset for image classification.

Features:

- Images of handwritten digits (0–9)
- Each image is grayscale of size **28 × 28 pixels**

Target Variable:

- Digit label (0 to 9)

Dataset Size:

- Training set: 60,000 images
- Testing set: 10,000 images

Characteristics:

- Image data (pixel values 0–255)
- Balanced dataset
- Suitable for CNN models

3. Mathematical Formulation of CNN

A CNN consists of convolution, pooling, and fully connected layers.

Convolution Operation:

$$Feature\ Map = Input * Kernel + Bias$$Feature Map=Input∗Kernel+Bias

Activation Function (ReLU):

$$f(x) = max(0, x)$$f(x)=max(0,x)

Pooling (Max Pooling):

$$y = max(x)$$y=max(x)

Softmax Output:

$$P(y = i) = \frac{e^{z_i}}{\sum e^{z_j}}$$P(y=i)=∑ezjezi

Loss Function (Categorical Crossentropy):

$$L = -\sum y \log(\hat{y})$$L=−∑ylog(y^)

4. Algorithm Limitations

- Requires high computational power
- Needs large dataset for best performance
- Training time is high
- Difficult to interpret (black-box)
- Sensitive to hyperparameters

5. Methodology / Workflow

Steps followed:

1. Load dataset
2. Normalize pixel values
3. Reshape data for CNN input
4. Convert labels to categorical
5. Build CNN model
6. Compile model
7. Train model
8. Evaluate performance
9. Visualize results

Workflow:

Data Loading → Preprocessing → CNN Model → Training → Evaluation → Visualization

6. Performance Analysis

Metrics used:

- Accuracy
- Loss

Interpretation:

- High accuracy (~98–99%) indicates strong model performance
- Low loss indicates good learning

Expected Result:

- CNN performs very well on MNIST due to spatial feature extraction

7. Hyperparameter Tuning

Parameters tuned:

- Number of convolution layers
- Number of filters
- Kernel size
- Batch size
- Epochs
- Optimizer (Adam, SGD)

Impact:

- More filters → better feature extraction
- More epochs → better learning (risk of overfitting)
- Batch size affects training speed
-

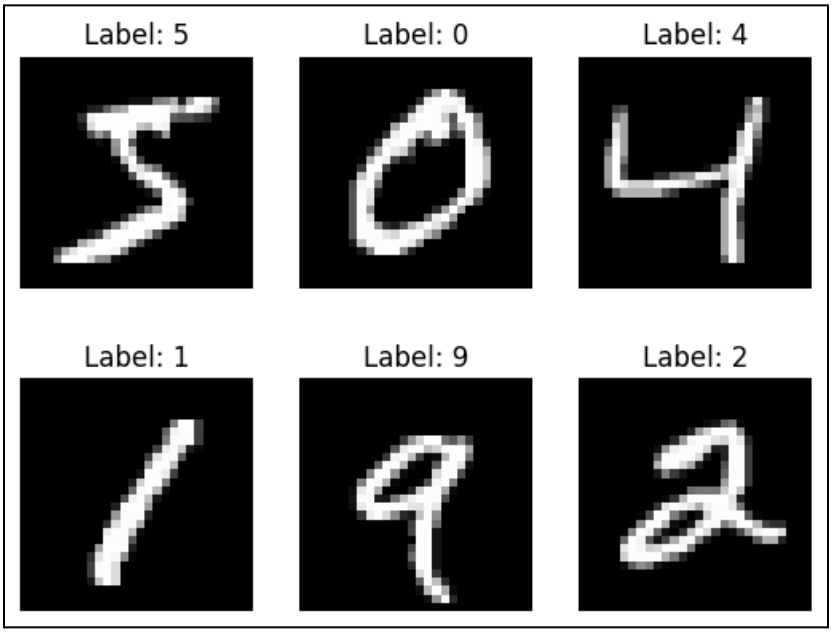
Code

```
# ================================

# CNN ON MNIST DATASET (FULL CODE)

# ================================

# Step 1: Install TensorFlow (only if needed)

!pip install tensorflow

# Step 2: Import Libraries

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras import datasets, layers, models

from tensorflow.keras.utils import to_categorical

from sklearn.metrics import classification_report, confusion_matrix
```

```
# Step 3: Load Dataset
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()
# Step 4: Explore Dataset
print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
# Display sample images
for i in range(6):
    plt.subplot(2,3,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title("Label: " + str(y_train[i]))
    plt.axis('off')
plt.show()
# Step 5: Data Preprocessing
# Normalize pixel values (0 to 1)
X_train = X_train / 255.0
X_test = X_test / 255.0
# Reshape data to include channel dimension
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
# One-hot encoding for labels
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
# Step 6: Build CNN Model
model = models.Sequential()
# Convolution Layer 1
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))
# Convolution Layer 2
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
# Convolution Layer 3
model.add(layers.Conv2D(64, (3,3), activation='relu'))
# Flatten layer
model.add(layers.Flatten())
# Fully connected layer
model.add(layers.Dense(64, activation='relu'))
# Output layer
model.add(layers.Dense(10, activation='softmax'))
# Step 7: Compile Model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Show model summary
model.summary()
# Step 8: Train Model
history = model.fit(
    X_train, y_train_cat,
    epochs=5,
    batch_size=64,
    validation_data=(X_test, y_test_cat)
)
# Step 9: Evaluate Model
test_loss, test_accuracy = model.evaluate(X_test, y_test_cat)
print("\nTest Accuracy:", test_accuracy)


# Step 10: Predictions
```

```
y_pred_probs = model.predict(X_test)

y_pred = np.argmax(y_pred_probs, axis=1)

# Step 11: Evaluation Metrics

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Step 12: Plot Accuracy Graph

plt.plot(history.history['accuracy'])

plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')

plt.xlabel('Epochs')

plt.ylabel('Accuracy')

plt.legend(['Train', 'Validation'])

plt.show()

# Step 13: Plot Loss Graph

plt.plot(history.history['loss'])

plt.plot(history.history['val_loss'])

plt.title('Model Loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend(['Train', 'Validation'])

plt.show()

# Step 14: Display Predictions

for i in range(6):

    plt.subplot(2,3,i+1)

    plt.imshow(X_test[i].reshape(28,28), cmap='gray')

    plt.title(f"Pred: {y_pred[i]}, Actual: {y_test[i]}")

    plt.axis('off')

plt.show()
```

Output



Harsh Kumar **D15B/65**

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dense_1 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)
Trainable params: 93,322 (364.54 KB)
Non-trainable params: 0 (0.00 B)

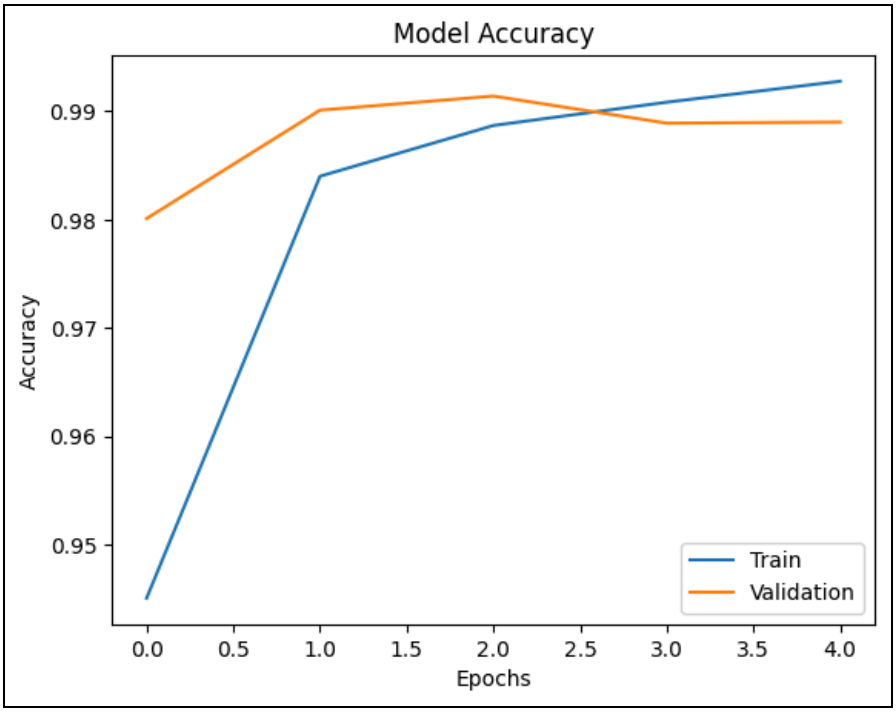
Confusion Matrix:

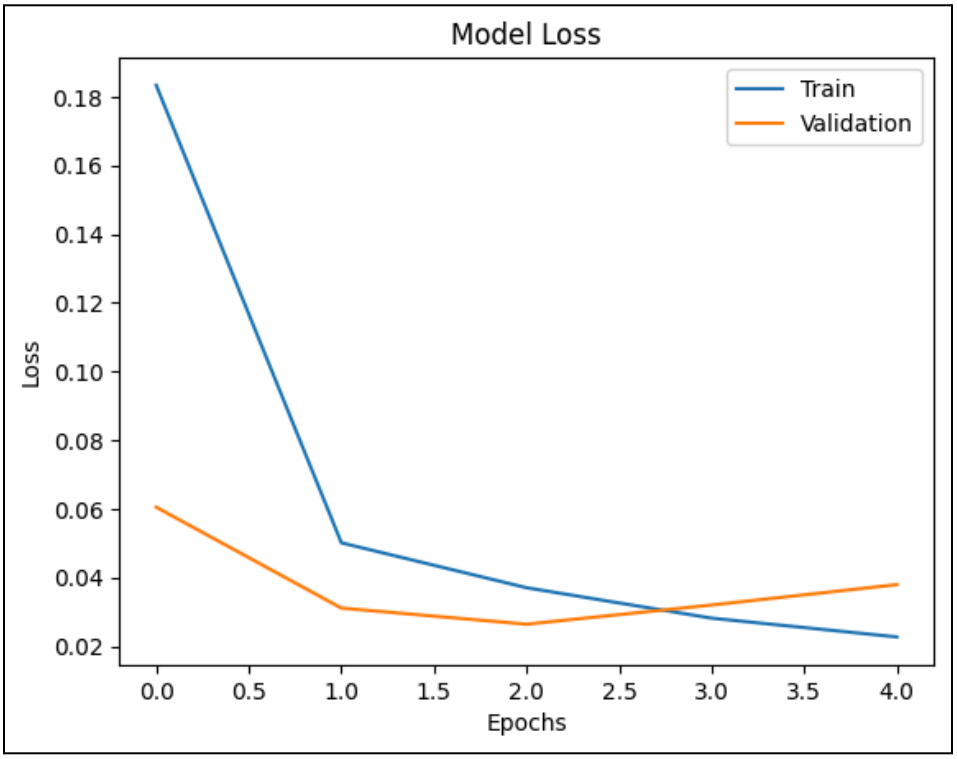
```
[[ 978    0    0    0    0    0    0    1    1    0]
 [   0 1130    0    1    1    0    0    0    2    1]
 [   2    1 1024    2    0    0    0    3    0    0]
 [   0    0    1 1006    0    3    0    0    0    0]
 [   0    0    0    0  973    0    0    0    1    8]
 [   3    0    0   10    0  876    1    0    2    0]
 [  12    1    1    0    5    4  932    0    3    0]
 [   0    3    3    1    0    0    0 1017    1    3]
 [   4    0    2    2    0    0    0    0  965    1]
 [   3    1    0    3    1    4    0    2    6  989]]
```

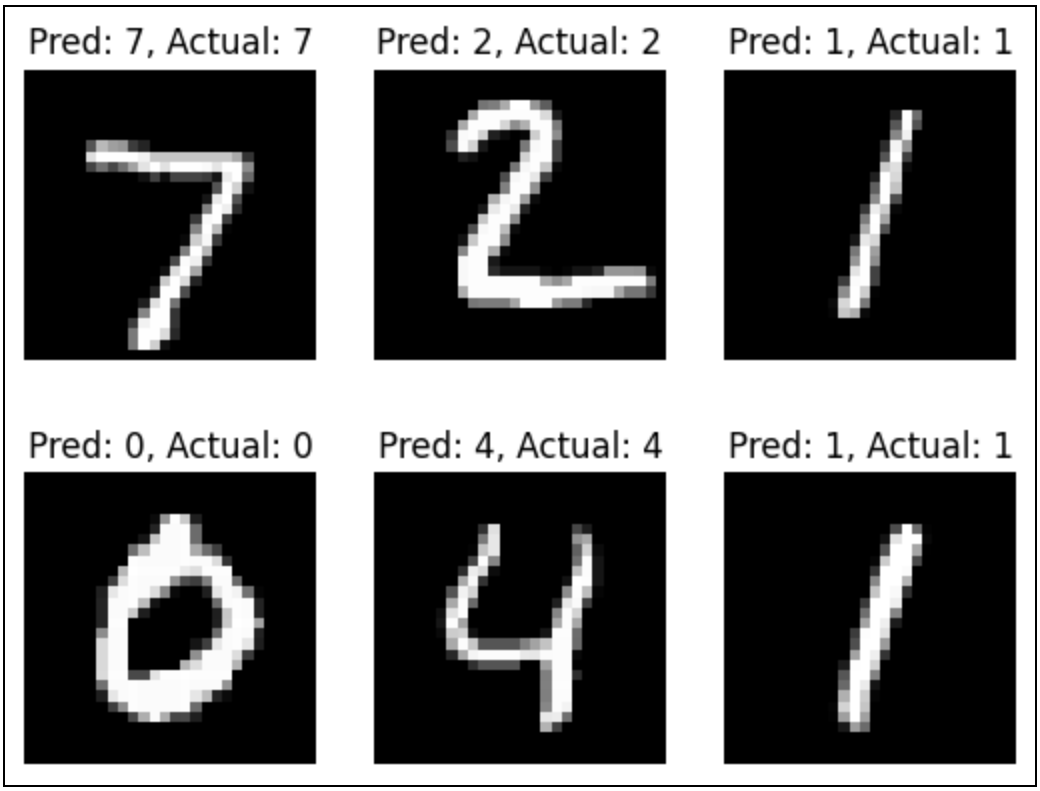
Classification Report:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	980
1	0.99	1.00	1.00	1135
2	0.99	0.99	0.99	1032
3	0.98	1.00	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.98	0.98	892
6	1.00	0.97	0.99	958
7	0.99	0.99	0.99	1028
8	0.98	0.99	0.99	974
9	0.99	0.98	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

Harsh Kumar **D15B/65**







Conclusion

The experiment successfully implemented a Convolutional Neural Network for image classification using the MNIST dataset. The CNN effectively extracted spatial features from images and achieved high accuracy. Preprocessing steps like normalization and reshaping improved model performance. The model demonstrated strong capability in recognizing handwritten digits. Overall, CNN proved to be highly efficient for image-based tasks.